

LINKED LISTS, ROOTED TREES & HASH TABLES

Lecture III for course in
Data Structures & Algorithms for AI

Lectures by Hannah Santa Cruz Baur. Based on *Introduction to Algorithms*, Cormen et al.

Linked lists

- We saw, in an array objects are arranged in linear order, determined by their indices.

0	1	2	3	4	5
1	4	2	5	3	6

Linked lists

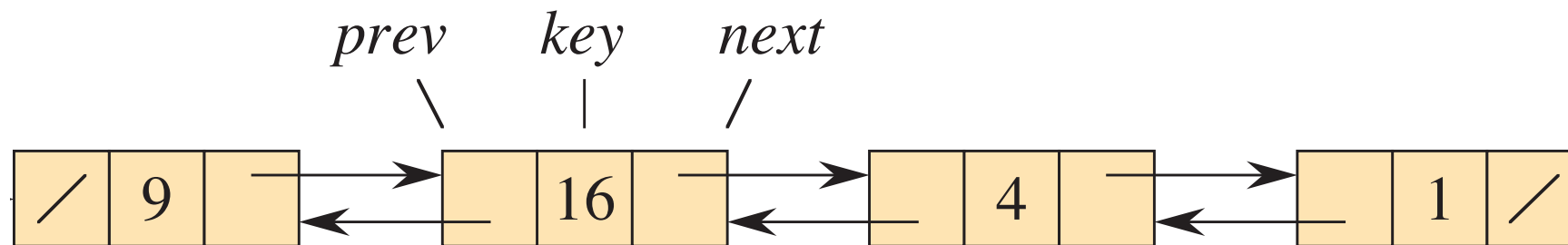
- We saw, in an array objects are arranged in linear order, determined by their indices.

0	1	2	3	4	5
1	4	2	5	3	6

- In a linked list, the objects are also stored in linear order, but this order is determined by pointers between the objects.

Linked lists

- We saw, in an array objects are arranged in linear order, determined by their indices.



- In a linked list, the objects are also stored in linear order, but this order is determined by pointers between the objects.
- A **doubly linked list** will have objects with three attributes: a *key* and two pointers, *prev* and *next*.

Searching a linked list

- To find the first element with *key* k in the list L , run:

LIST-SEARCH(L, k)

```
1   $x = L.head$   
2  while  $x \neq \text{NIL}$  and  $x.key \neq k$   
3       $x = x.next$   
4  return  $x$ 
```

Searching a linked list

- To find the first element with *key* k in the L , run:

```
LIST-SEARCH( $L, k$ )  
1   $x = L.head$   
2  while  $x \neq \text{NIL}$  and  $x.key \neq k$   
3       $x = x.next$   
4  return  $x$ 
```

Notice $L.head$ refers to the first element of the list, it's *prev* attribute is NIL.

Similarly the last element of the list, is called *tail*, and it's *next* attribute is NIL.

Inserting into a linked list

- To add an element, x , to the front of the list L , run:

LIST-PREPEND(L, x)

1 $x.next = L.head$

2 $x.prev = \text{NIL}$

3 **if** $L.head \neq \text{NIL}$

4 $L.head.prev = x$

5 $L.head = x$

Inserting into a linked list

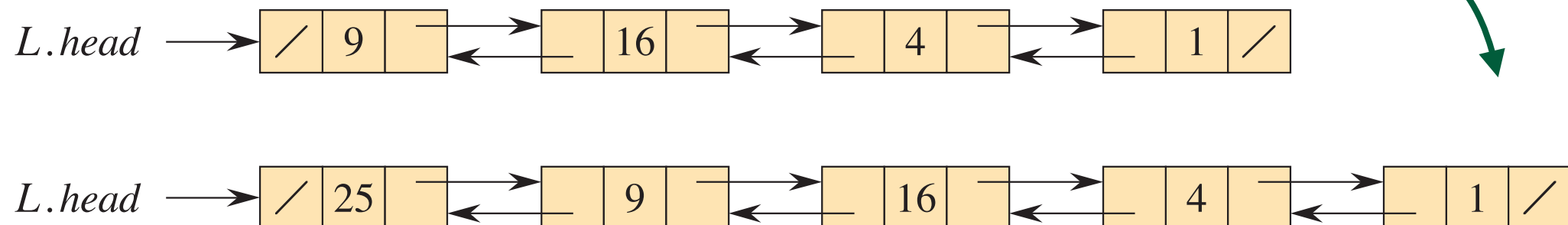
- To add an element, x , to the front of the list L , run:

LIST-PREPEND(L, x)

```
1   $x.next = L.head$ 
2   $x.prev = NIL$ 
3  if  $L.head \neq NIL$ 
4       $L.head.prev = x$ 
5   $L.head = x$ 
```

for $x.key=25$

LIST-PREPEND(L, x)



Inserting into a linked list

- To add an element, x , to the front of the list L , run:

LIST-PREPEND(L, x)

```
1   $x.next = L.head$   
2   $x.prev = \text{NIL}$   
3  if  $L.head \neq \text{NIL}$   
4       $L.head.prev = x$   
5   $L.head = x$ 
```

What happens if the list is empty
when you call prepend?

Inserting into a linked list

- To add an element, x , to the front of the list L , run:

LIST-PREPEND(L, x)

```
1   $x.next = L.head$ 
2   $x.prev = NIL$ 
3  if  $L.head \neq NIL$ 
4       $L.head.prev = x$ 
5   $L.head = x$ 
```

To add an element, x , following an element, y , in the list, run:

LIST-INSERT(x, y)

```
1   $x.next = y.next$ 
2   $x.prev = y$ 
3  if  $y.next \neq NIL$ 
4       $y.next.prev = x$ 
5   $y.next = x$ 
```

Inserting into a linked list

- To add an element, x , to the front of the list L , run:

LIST-PREPEND(L, x)

```
1   $x.next = L.head$ 
2   $x.prev = NIL$ 
3  if  $L.head \neq NIL$ 
4       $L.head.prev = x$ 
5   $L.head = x$ 
```

To add an element, x , following an element, y , in the list, run:

LIST-INSERT(x, y)

```
1   $x.next = y.next$ 
2   $x.prev = y$ 
3  if  $y.next \neq NIL$ 
4       $y.next.prev = x$ 
5   $y.next = x$ 
```

Notice the actual list is never referenced in the INSERT function.

Inserting into a linked list

- To add an element, x , to the front of the list L , run:

LIST-PREPEND(L, x)

```
1   $x.next = L.head$ 
2   $x.prev = NIL$ 
3  if  $L.head \neq NIL$ 
4       $L.head.prev = x$ 
5   $L.head = x$ 
```

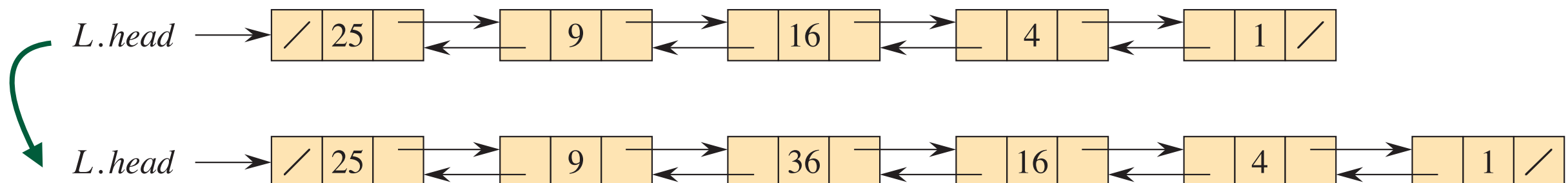
To add an element, x , following an element, y , in the list, run:

LIST-INSERT(x, y)

```
1   $x.next = y.next$ 
2   $x.prev = y$ 
3  if  $y.next \neq NIL$ 
4       $y.next.prev = x$ 
5   $y.next = x$ 
```

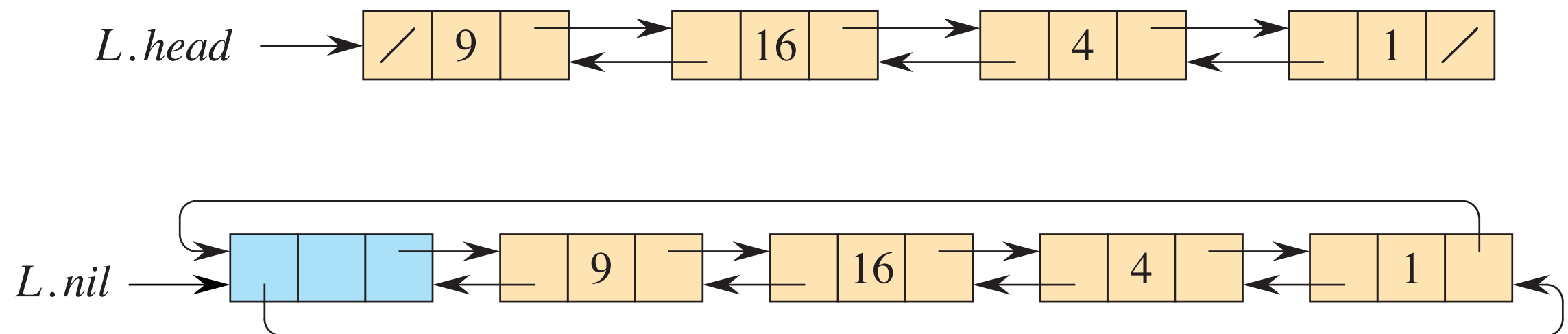
for $x.key=36, y.key=9$

LIST-INSERT(x, y)



Sentinels

- It is often useful to add a dummy object to lists, making them **circular**. We call this object a **sentinel**, and have it represent NIL.



- Instead of referencing NIL, we now call the sentinel, $L.nil$.
- To reference the *head*, we call $L.nil.next$, and to reference the *tail* we call $L.nil.prev$.

Inserting into a linked list

- We can now simplify the insertion process into a single function, as we no longer have an attribute *L.head*.

LIST-INSERT'(*x*, *y*)

1 *x.next* = *y.next*

2 *x.prev* = *y*

3 *y.next.prev* = *x*

4 *y.next* = *x*

Inserting into a linked list with a sentinel

- We can now simplify the insertion process into a single function, as we no longer have an attribute *L.head*.

LIST-INSERT'(*x*, *y*)

1 *x.next* = *y.next*

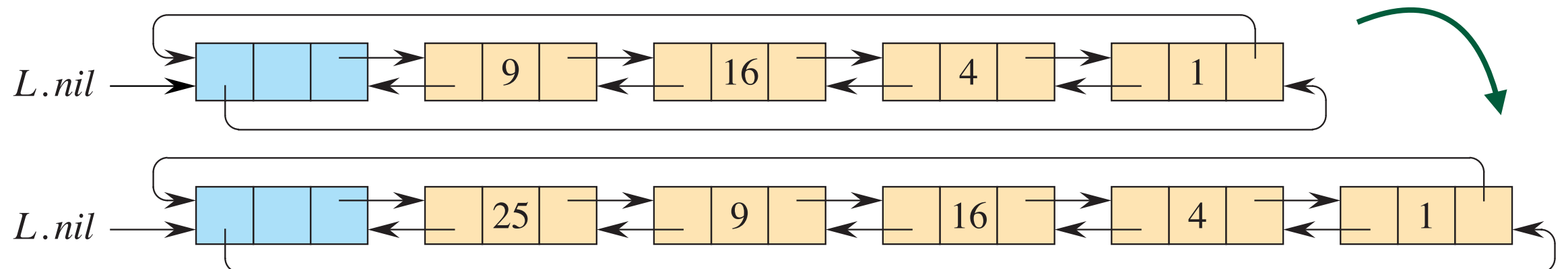
2 *x.prev* = *y*

3 *y.next.prev* = *x*

4 *y.next* = *x*

for *x.key*=25

LIST-INSERT(*x*, *L.nil*)



Deleting from a linked list

- Deleting an element is easy as splicing.

LIST-DELETE (x)

1 $x.\text{prev}.\text{next} = x.\text{next}$

2 $x.\text{next}.\text{prev} = x.\text{prev}$

Deleting from a linked list

- Deleting an element is easy as splicing.

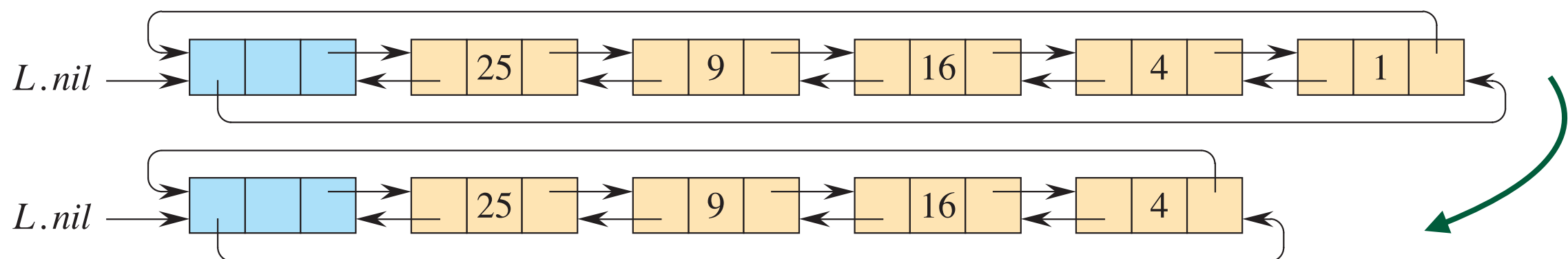
LIST-DELETE (x)

1 $x.prev.next = x.next$

2 $x.next.prev = x.prev$

for $x.key=1$

LIST-DELETE(x)



Deleting from a linked list

- Deleting an element is easy as splicing.

LIST-DELETE (x)

1 $x.prev.next = x.next$

2 $x.next.prev = x.prev$

What happens if you call *LIST-DELETE*(*L.nil*)?

Searching a linked list with a sentinel

- We can also use the sentinel to temporarily store the value of the object we are searching.

Searching a linked list with a sentinel

- We can also use the sentinel to temporarily store the value of the object we are searching.
- This way guaranteeing the item is found, and eliminating one of the comparison checks of the SEARCH function.

LIST-SEARCH(L, k)

```
1   $x = L.head$   
2  while  $x \neq \text{NIL}$  and  $x.key \neq k$   
3       $x = x.next$   
4  return  $x$ 
```

Searching a linked list with a sentinel

- We can also use the sentinel to temporarily store the value of the object we are searching.
- This way guaranteeing the item is found, and eliminating one of the comparison checks of the SEARCH function.
- This speeds up the algorithm!

LIST-SEARCH(L, k)

```
1   $x = L.head$   
2  while  $x \neq \text{NIL}$  and  $x.key \neq k$   
3       $x = x.next$   
4  return  $x$ 
```



LIST-SEARCH'(L, k)

```
1   $L.nil.key = k$   
2   $x = L.nil.next$   
3  while  $x.key \neq k$   
4       $x = x.next$   
5  if  $x == L.nil$   
6      return NIL  
7  else return  $x$ 
```

Searching a linked list with a sentinel

- We can also use the sentinel to temporarily store the value of the object we are searching.
- This way guaranteeing the item is found, and eliminating one of the comparison checks of the SEARCH function.
- This speeds up the algorithm!

Storing the value
temporarily in the sentinel

LIST-SEARCH'(L, k)

```
1   $L.nil.key = k$ 
2   $x = L.nil.next$ 
3  while  $x.key \neq k$ 
4       $x = x.next$ 
5  if  $x == L.nil$ 
6      return NIL
7  else return  $x$ 
```

Searching a linked list with a sentinel

- We can also use the sentinel to temporarily store the value of the object we are searching.
- This way guaranteeing the item is found, and eliminating one of the comparison checks of the SEARCH function.
- This speeds up the algorithm!

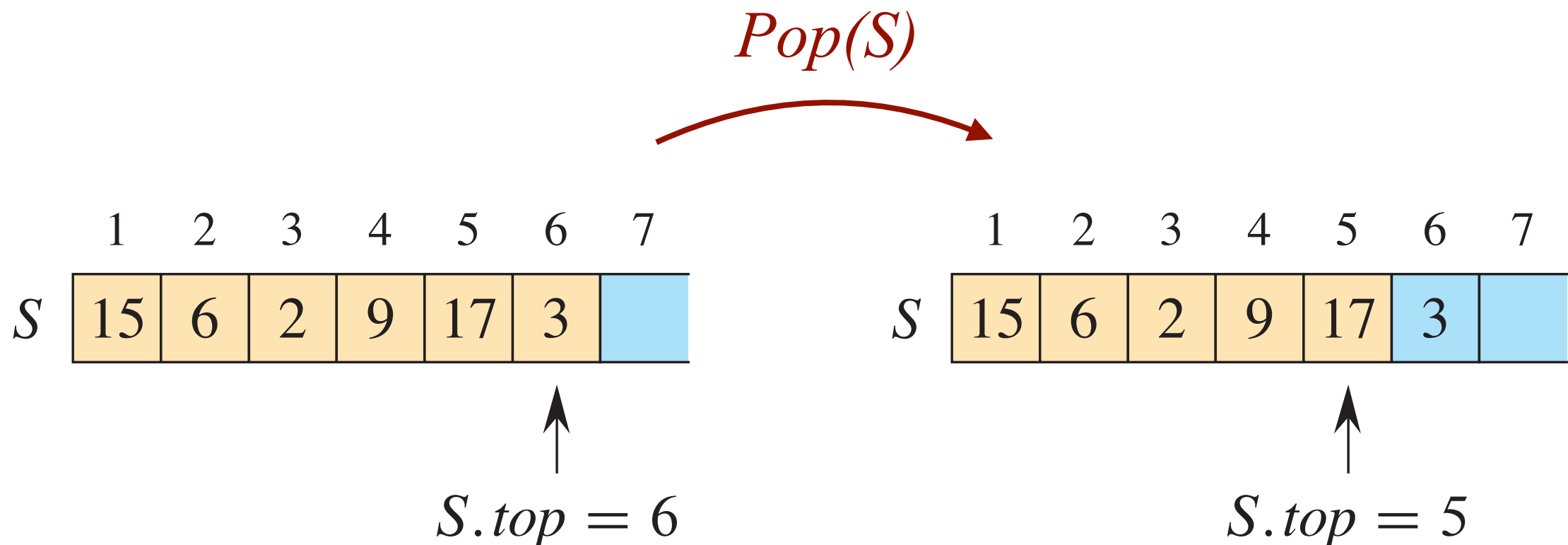
The search begins right after the sentinel

If the value is not found until the sentinel is reached, the value was not originally in the list

```
LIST-SEARCH'(L, k)
1  L.nil.key = k
2  x = L.nil.next
3  while x.key ≠ k
4      x = x.next
5  if x == L.nil
6      return NIL
7  else return x
```


Exercise

- Recall a **stack** is a dynamic set, where the delete operation eliminates the most recently inserted element.
- We saw the implementation of a stack using arrays, now implement a stack using a singly linked list.



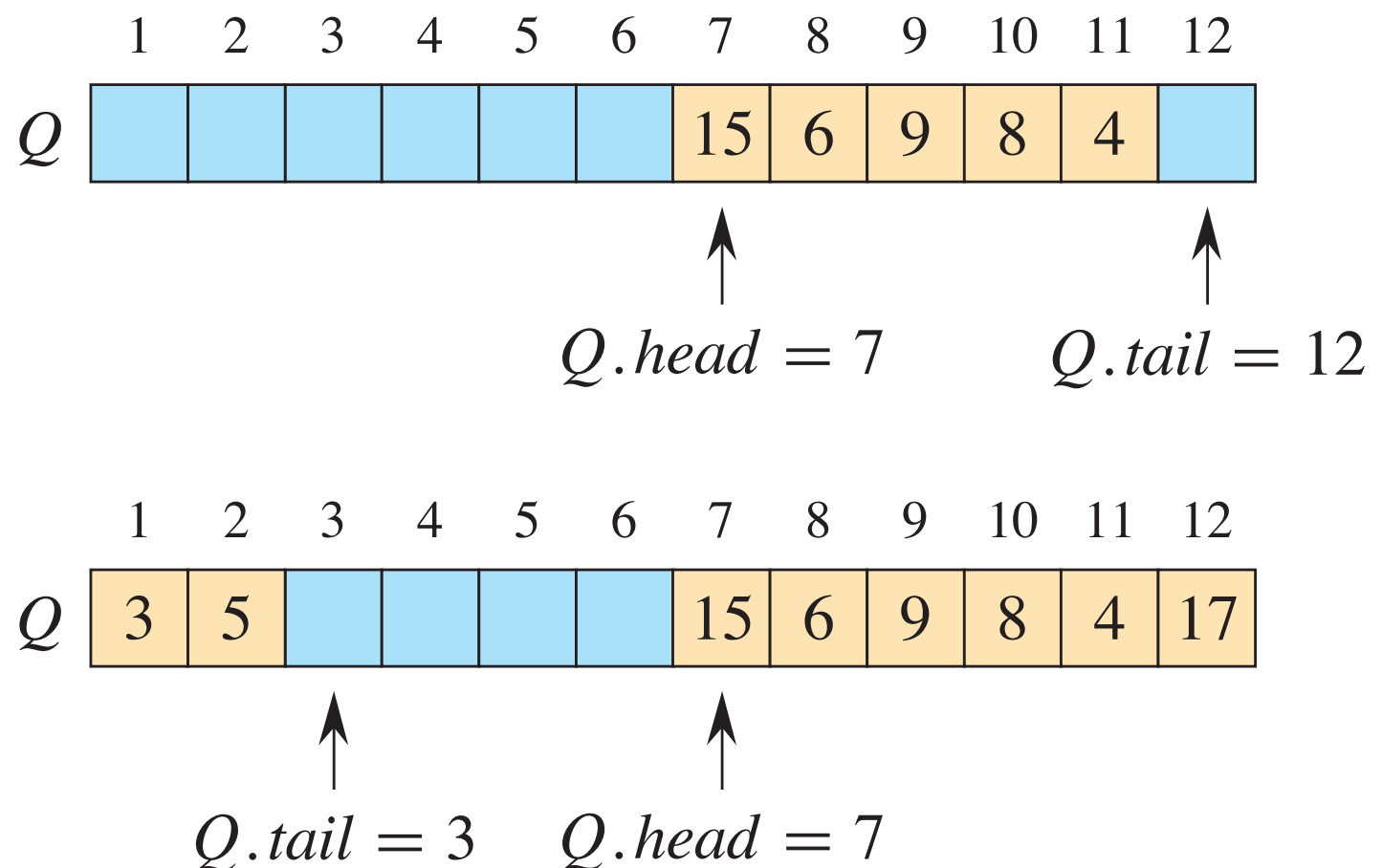
Homework (due with Assignment 1)

- A **queue** is a dynamic set, where the delete operation eliminates the element which has been stored the longest.
- We saw the implementation of a queue using arrays, now implement a queue using linked lists. Is it easier to implement with or without a sentinel?

Enqueue(Q, 17)

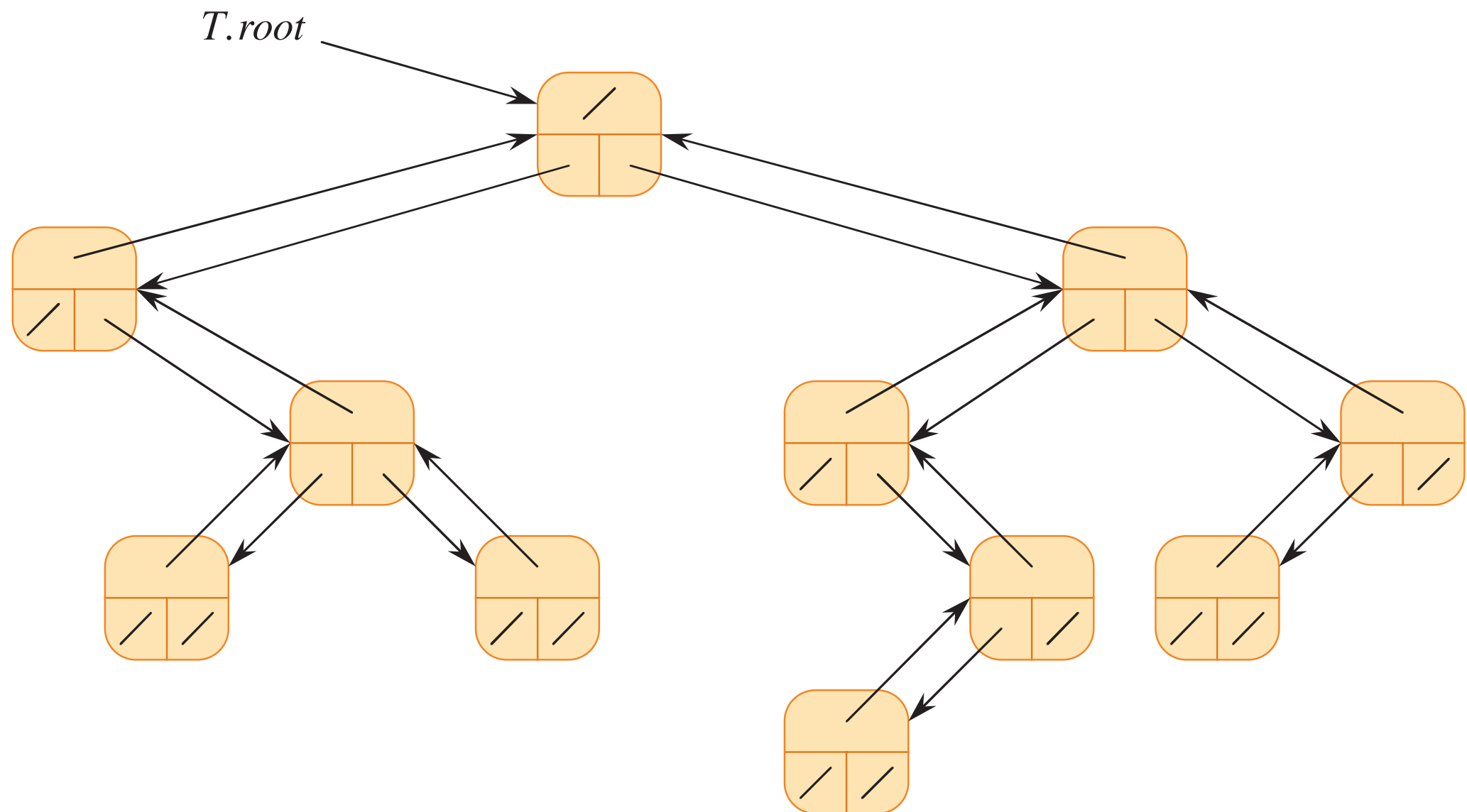
Enqueue(Q, 3)

Enqueue(Q, 5)



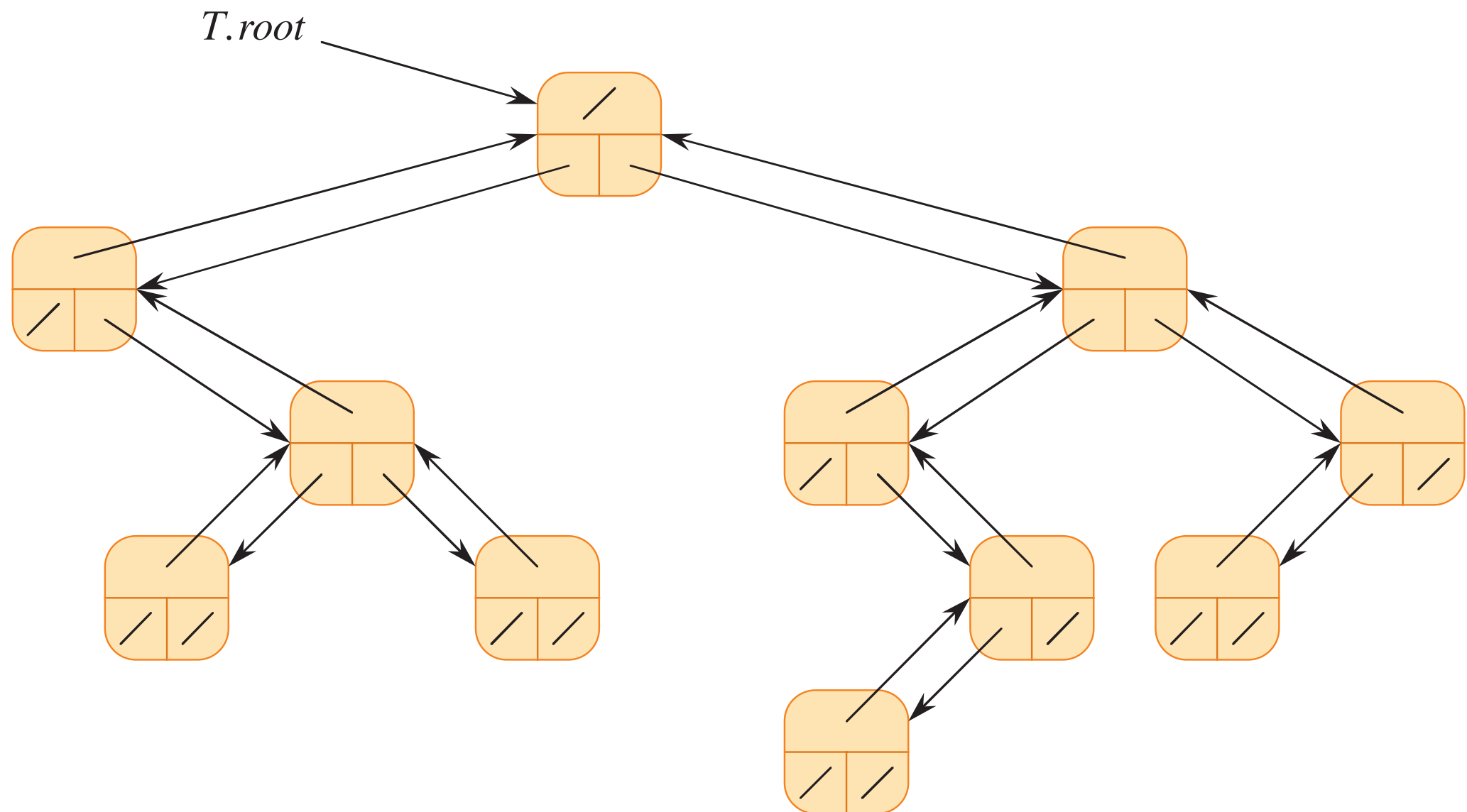
Rooted trees

- Let us now consider linked data structures, representing more than linear relationships.



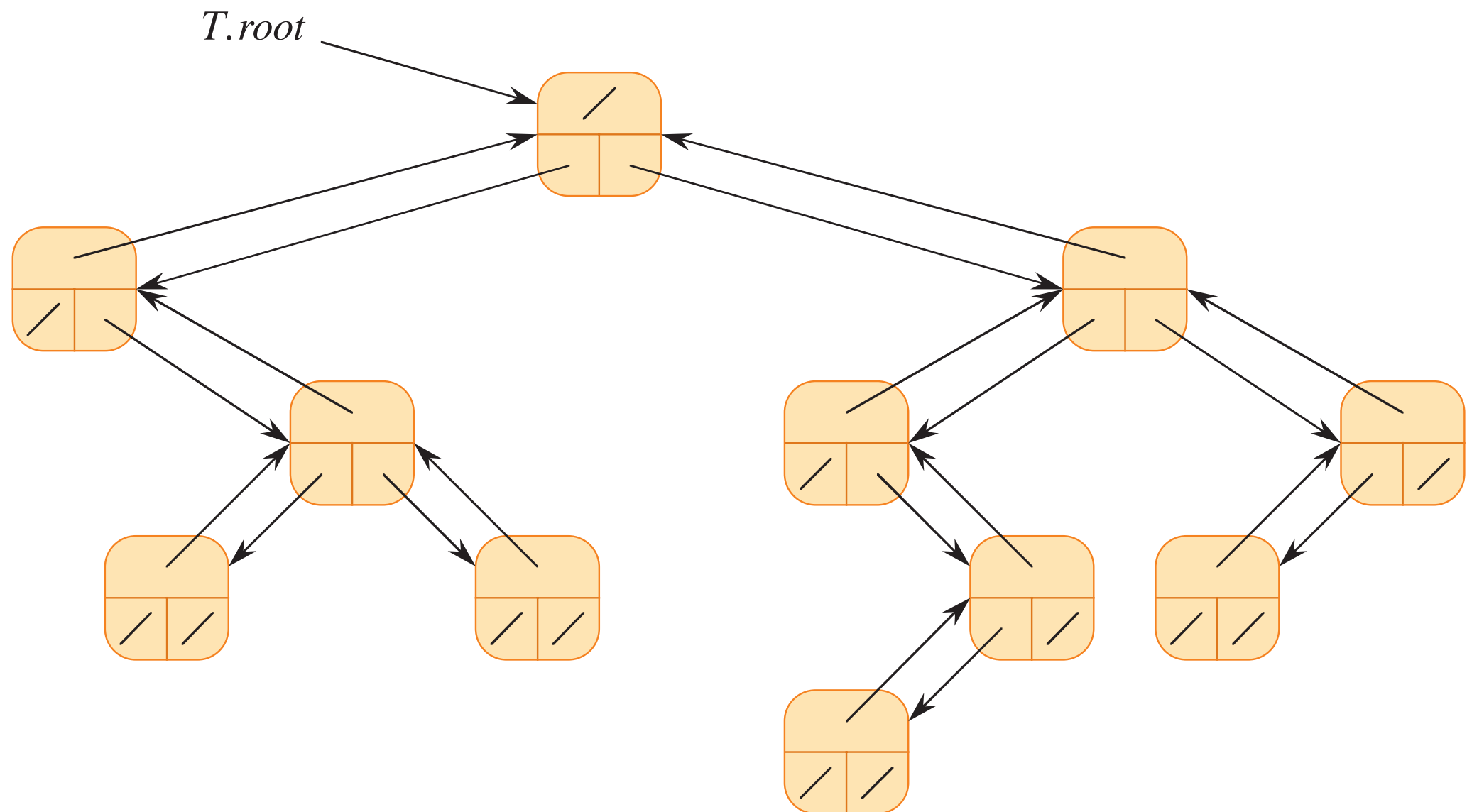
Rooted trees

- A binary tree has objects with a *key* attribute, along with pointer attributes, p (top), (lower) *left* and (lower) *right*.



Exercise

- Write a recursive procedure that prints out the key for each node in a n -node binary tree.



Exercise

- Write a recursive procedure that prints out the key for each node in a n -node binary tree.

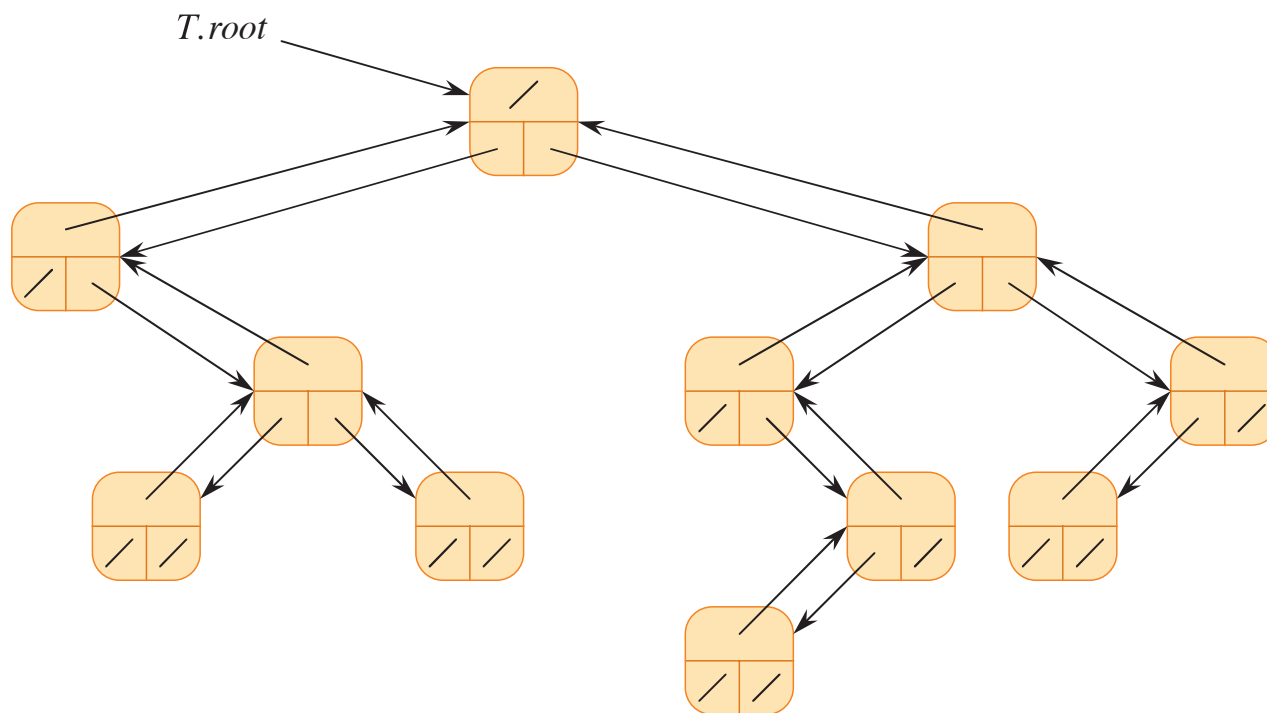
PRINT-TREE-WALK(x)

If $x \neq \text{NIL}$

PRINT-TREE-WALK($x.\text{left}$)

print($x.\text{key}$)

PRINT-TREE-WALK($x.\text{right}$)



Exercise

- Write a recursive procedure that prints out the key for each node in a n -node binary tree.

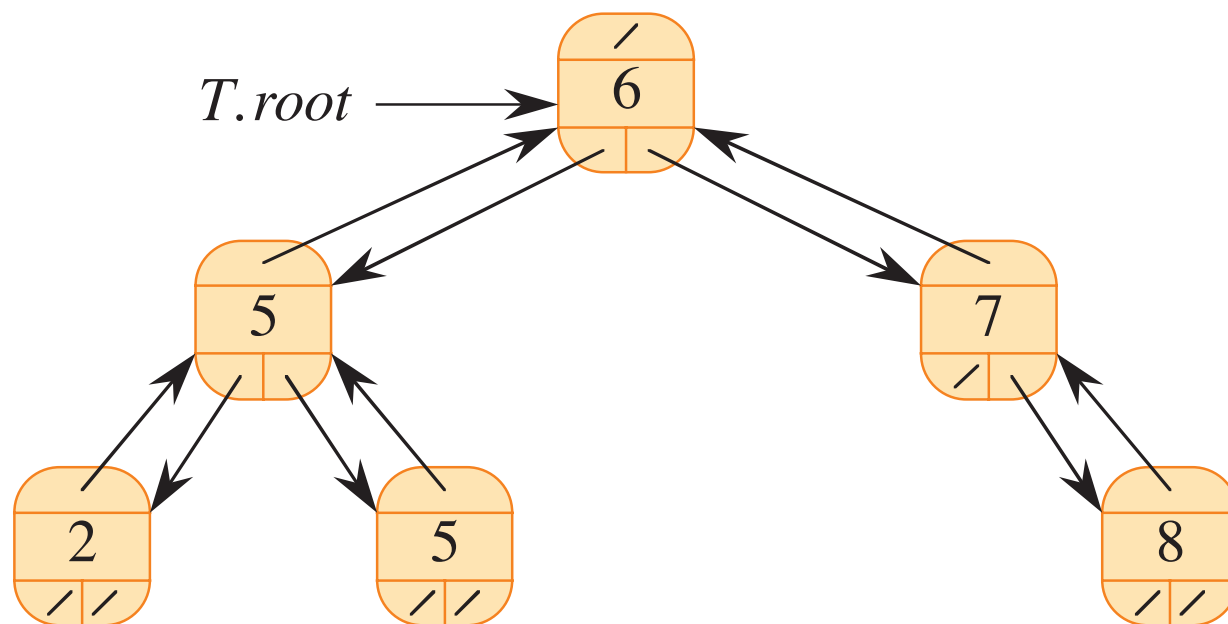
PRINT-TREE-WALK(x)

If $x \neq \text{NIL}$

PRINT-TREE-WALK($x.\text{left}$)

print($x.\text{key}$)

PRINT-TREE-WALK($x.\text{right}$)

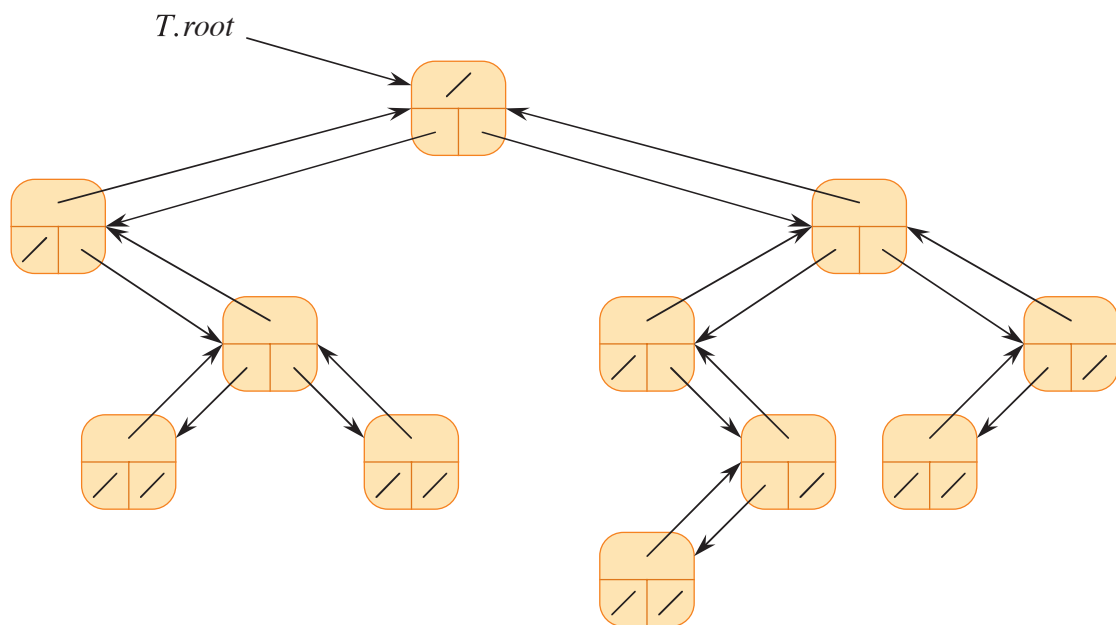


2, 5, 5, 6, 7, 8

- What order would the keys be printed in?

Exercise

- Write a recursive procedure that prints out the key for each node in a n -node binary tree.



- Write a non recursive version, using a stack as an auxiliary structure.

Exercise

- Write a recursive procedure that prints out the key for each node in a n -node binary tree.
- PRINT-TREE-WALK'(x)**

PRINT-TREE-WALK'(x)

Define stack S

$$y=x$$

While $y \neq \text{NIL}$ or $\text{ISEMPTY}(S) = \text{False}$

While $y \neq \text{NIL}$

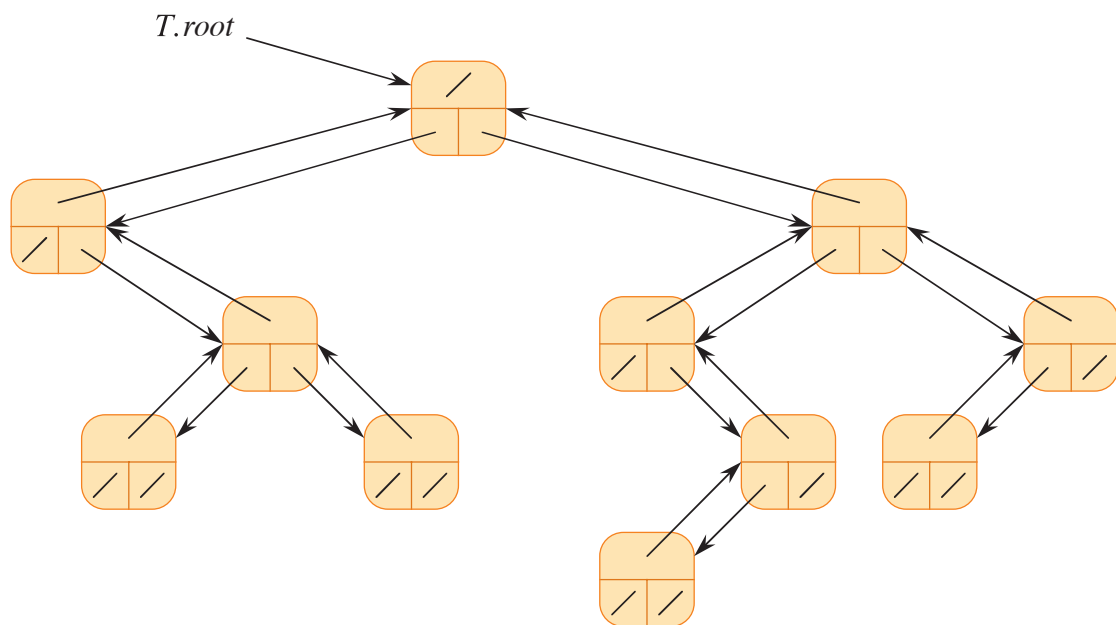
PUSH(S, y)

y = *y.left*

$$y = \text{POP}(S)$$

Print(*y.key*)

y=y.right



- Write a non recursive version, using a stack as an auxiliary structure.

Exercise

- What order are the keys printed in?

PRINT-TREE-WALK'(x)

Define stack S

$y = x$

While $y \neq \text{NIL}$ or $\text{ISEMPTY}(S) = \text{False}$

While $y \neq \text{NIL}$

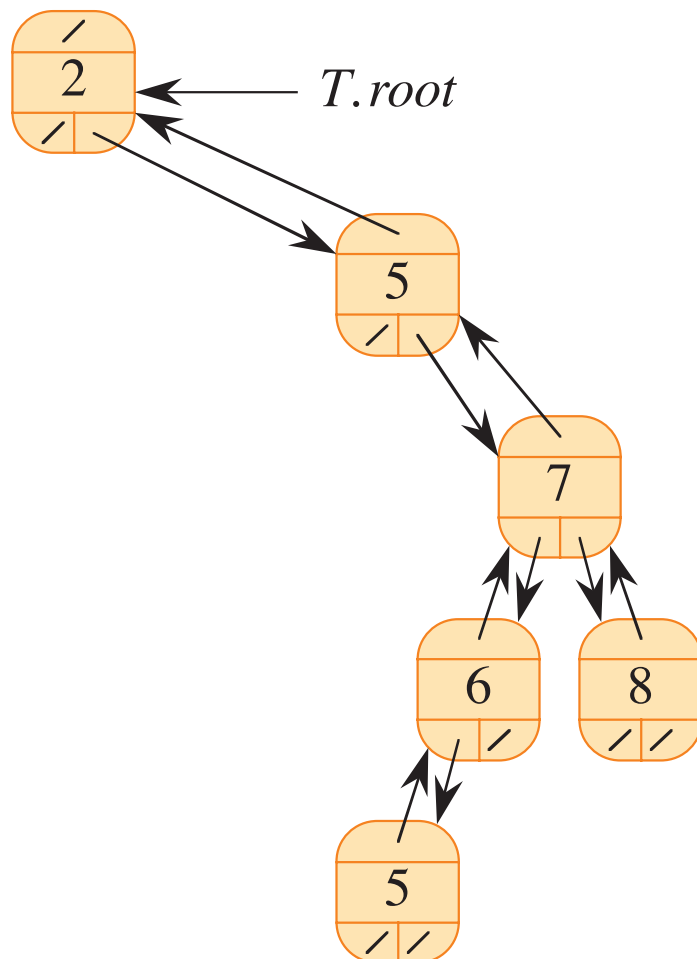
PUSH(S, y)

$y = y.\text{left}$

$y = \text{POP}(S)$

Print($y.\text{key}$)

$y = y.\text{right}$



- Write a non recursive version, using a stack as an auxiliary structure.

Exercise

- What order are the keys printed in? $2, 5, 5, 6, 7, 8$;)

PRINT-TREE-WALK'(x)

Define stack S

$y = x$

While $y \neq \text{NIL}$ or $\text{ISEMPTY}(S) = \text{False}$

While $y \neq \text{NIL}$

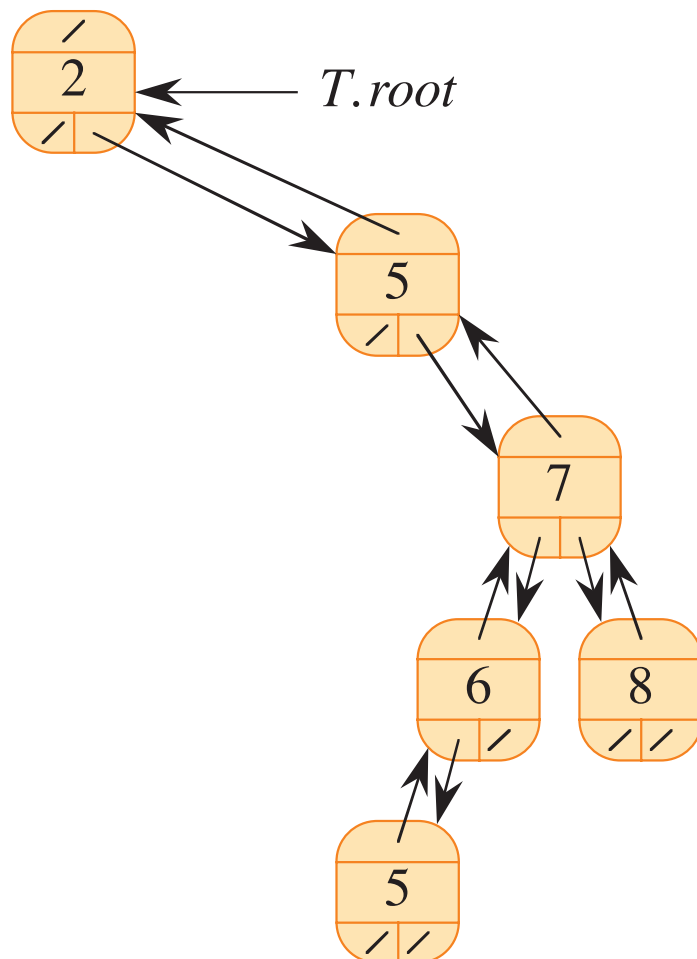
PUSH(S, y)

$y = y.\text{left}$

$y = \text{POP}(S)$

Print($y.\text{key}$)

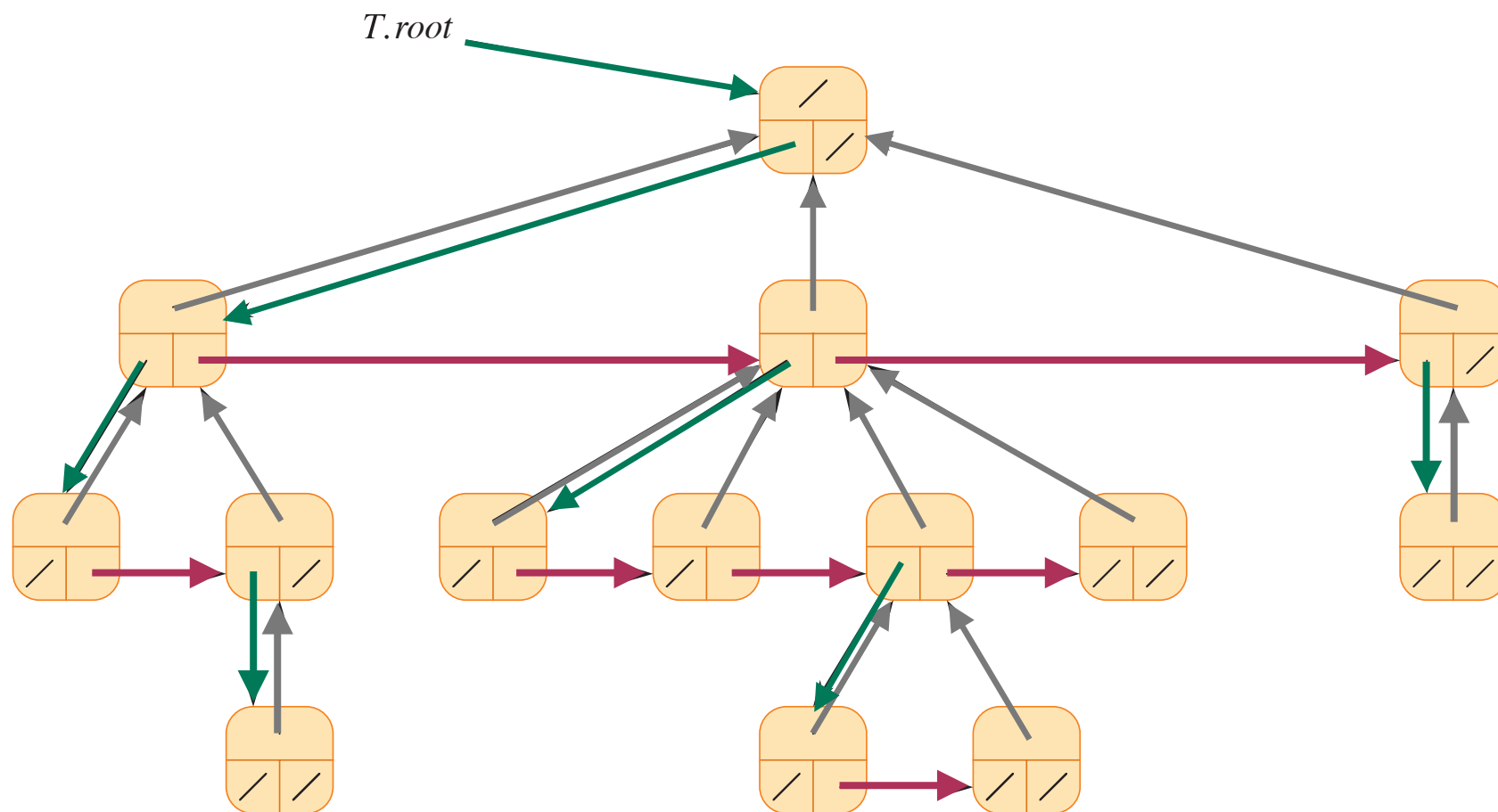
$y = y.\text{right}$



- Write a non recursive version, using a stack as an auxiliary structure.

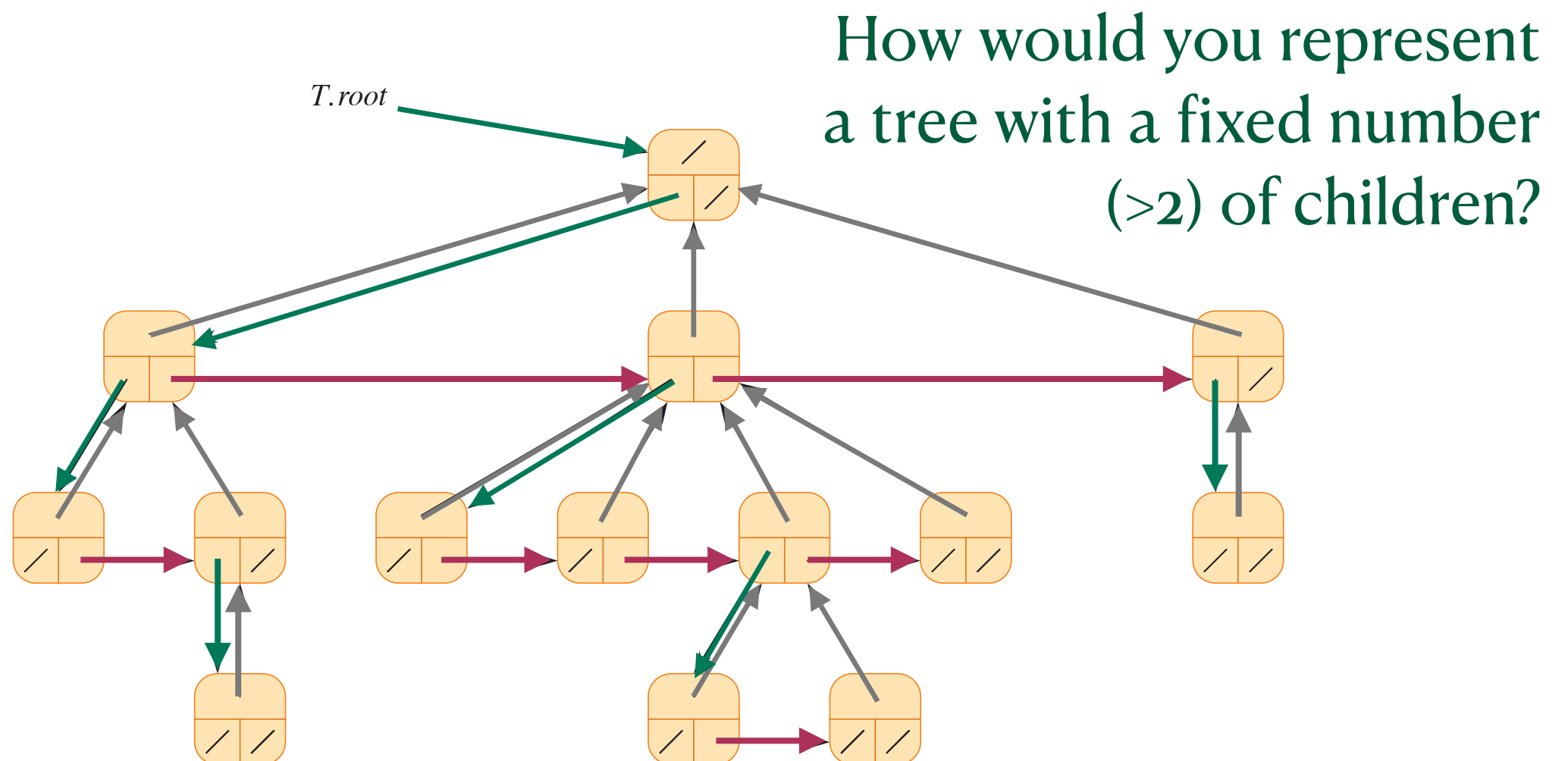
Rooted trees

- To represent a tree in which nodes have an arbitrary number of “children”, we define pointers to the parent - p , the “left-most” child - *left.child*, and the sibling immediately to the right - *right.sibling*.



Rooted trees

- To represent a tree in which nodes have an arbitrary number of “children”, we define pointers to the parent - p , the “left-most” child - *left.child*, and the sibling immediately to the right - *right.sibling*.



Dictionary data structures

- When the only operations required from a dynamic set are INSERT, SEARCH and DELETE, it can be most effective to implement a dictionary structure.

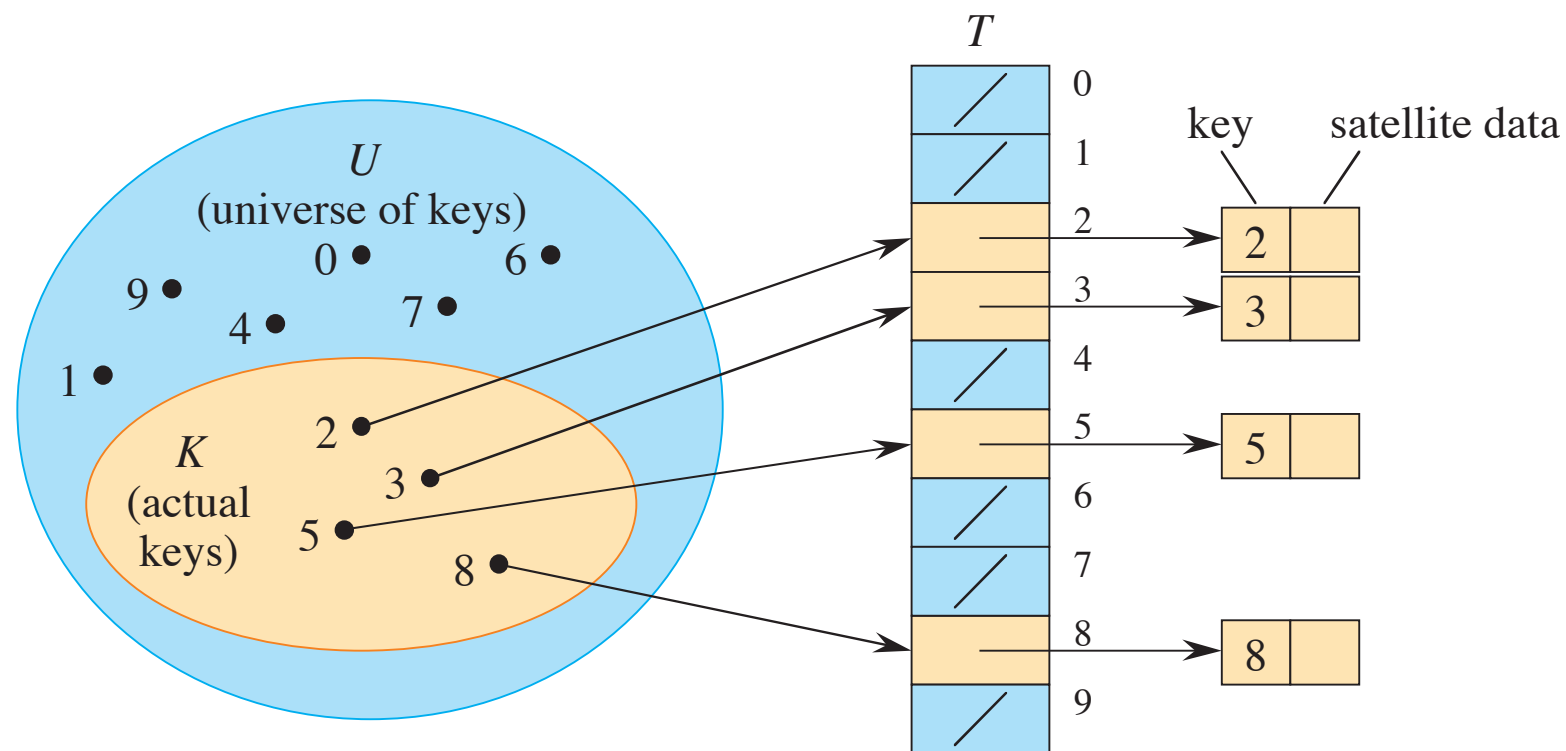
Dictionary data structures

- When the only operations required from a dynamic set are INSERT, SEARCH and DELETE, it can be most effective to implement a dictionary structure.

as opposed to operations such as
successor, predecessor, minimum, maximum

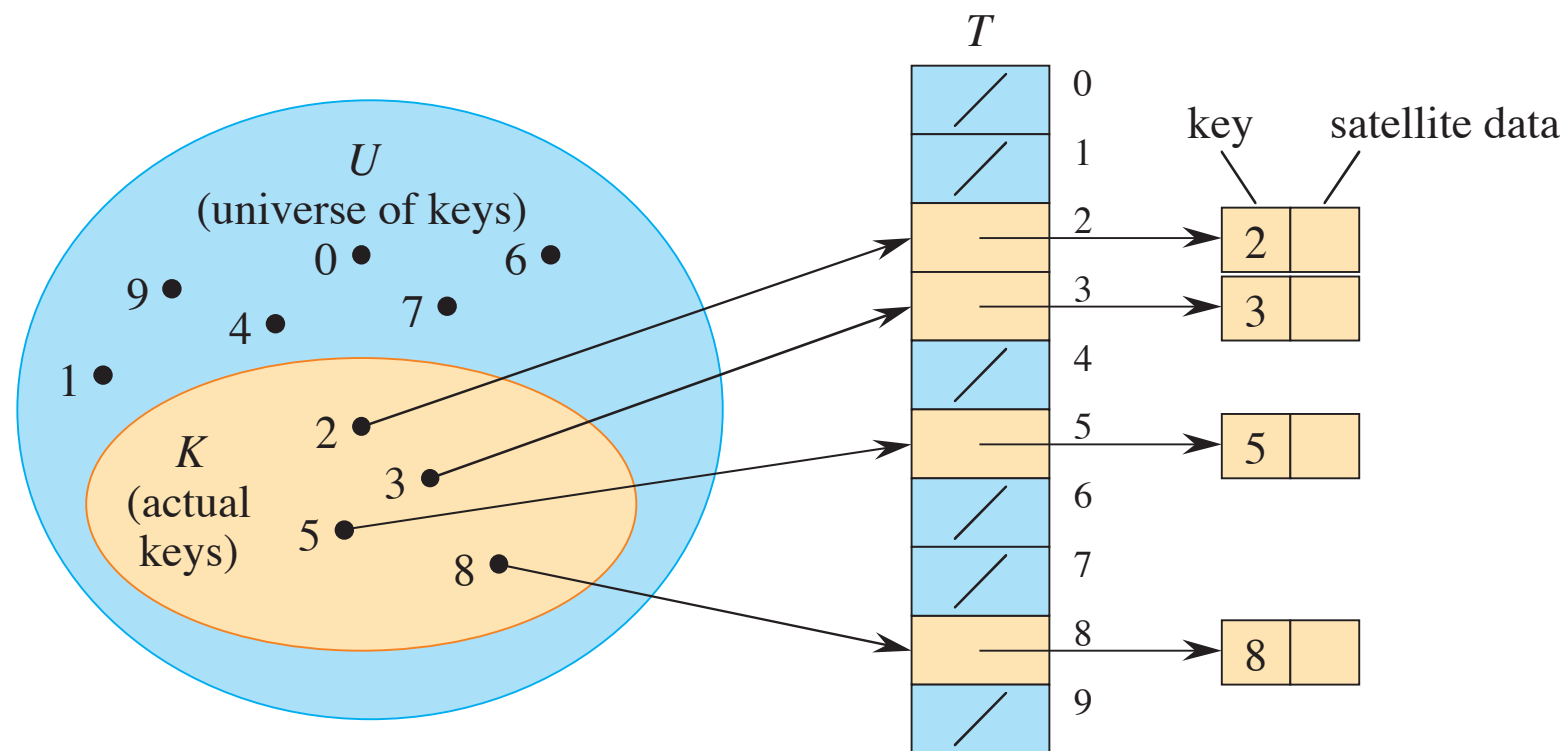
Dictionary data structures

- To implement a dictionary structure, we use an array in which each position corresponds to a key of the universe. Then finding a key and its associated satellite data, takes constant time.



Dictionary data structures

- To implement a dictionary structure, we use an array in which each position corresponds to a key of the universe. Then finding a key and its associated satellite data, takes constant time.



- This can however be impractical, if the universe of keys is very large with respect to the keys actually used.

Hash tables

- Hash tables are efficient dictionaries in which:
 - search time remain constant on average,
 - storage is proportional to the total keys actually used.

Hash tables

- Hash tables are efficient dictionaries in which:
 - search time remain constant on average,
 - storage is proportional to the total keys actually used.
- Instead of storing the element with *key* k , in position k , a **hash function**, is used to associate to it a position $h(k)$.
- The hash function maps the universe of keys, U , into the positions of the hash table $T[0:m-1]$:

$$h : U \rightarrow \{0, \dots, m - 1\}$$

Hash tables

- Hash tables are efficient dictionaries in which:
 - search time remain constant on average,
 - storage is proportional to the total keys actually used.
- Instead of storing the element with *key* k , in position k , a **hash function**, is used to associate to it a position $h(k)$.
- The hash function maps the universe of keys, U , into the positions of the hash table $T[0:m-1]$:

$$h : U \rightarrow \{0, \dots, m - 1\}$$

- The catch, is we have to watch out for collisions: when two keys map (hash) onto the same position.

Hash function example

- One simple hash function, would be the **division method**.

$$h(k) = k \bmod m$$

- Hashing by division is quite fast.

Hash function example

- One simple hash function, would be the **division method**.

$$h(k) = k \bmod m$$

- Hashing by division is quite fast. However it has no guarantees regarding collision avoidance - thus does not guarantee a good average case performance.

Hash function example

- One simple hash function, would be the **division method**.

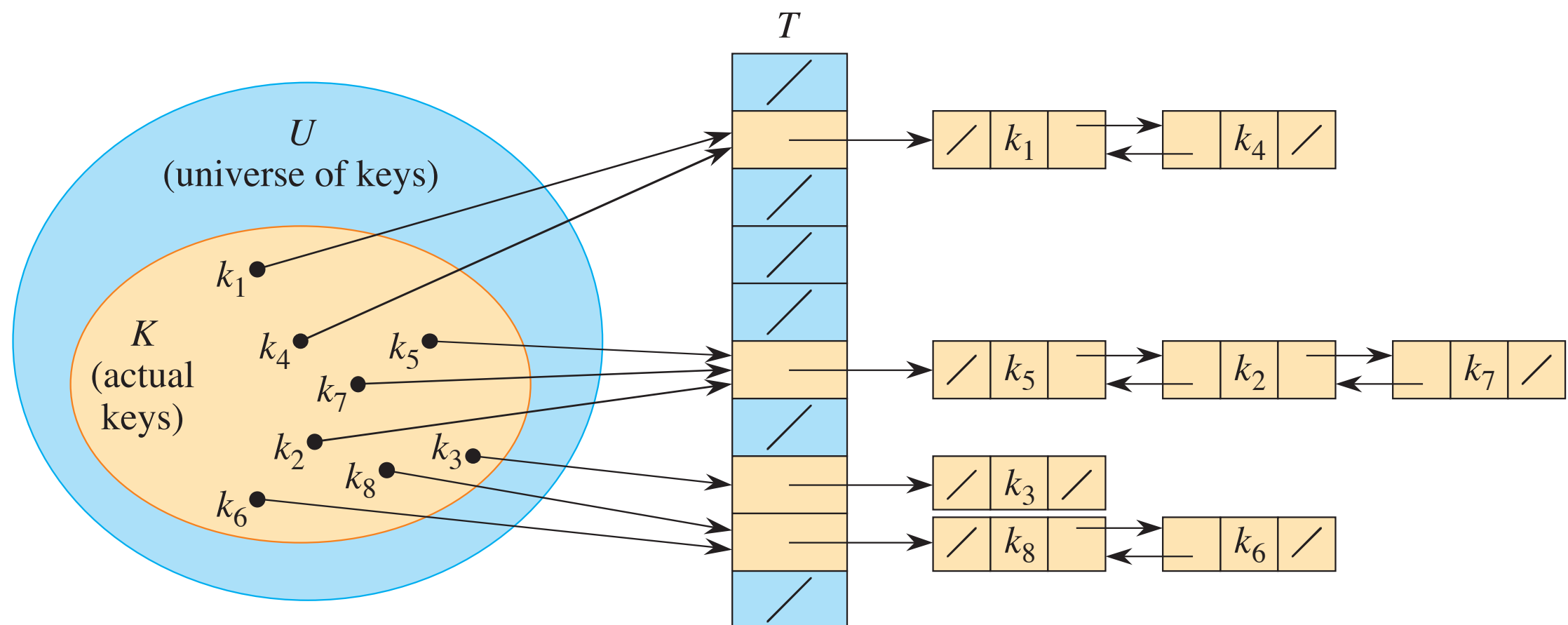
$$h(k) = k \bmod m$$

- Hashing by division is quite fast. However it has no guarantees regarding collision avoidance - thus does not guarantee a good average case performance.
- It also only works well when m is chosen to be a prime, and not too close to a power of 2.

Collision resolution

- The simplest collision resolution method, is chaining:

Each nonempty slot points to a linked list, and all elements which hash onto the same position go into that linked list.



Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

LIST-DELETE(x)

1 $x.prev.next = x.next$

2 $x.next.prev = x.prev$

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

If the lists are doubly linked, deletion takes constant time to run.

LIST-DELETE(x)

1 $x.prev.next = x.next$

2 $x.next.prev = x.prev$

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

LIST-SEARCH(L, k)

1 $L.nil.key = k$

2 $x = L.nil.next$

3 **while** $x.key \neq k$

4 $x = x.next$

5 **if** $x == L.nil$

6 **return** NIL

7 **else return** x

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

LIST-SEARCH(L, k)

1 $L.nil.key = k$

2 $x = L.nil.next$

3 **while** $x.key \neq k$

4 $x = x.next$

5 **if** $x == L.nil$

6 **return** NIL

7 **else return** x

The search operation has a worst case running time proportional to the list length.

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

LIST-PREPEND(L, x)

1 $x.next = L.head$

2 $x.prev = \text{NIL}$

3 **if** $L.head \neq \text{NIL}$

4 $L.head.prev = x$

5 $L.head = x$

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

LIST-PREPEND(L, x)

1 $x.next = L.head$

2 $x.prev = \text{NIL}$

3 **if** $L.head \neq \text{NIL}$

4 $L.head.prev = x$

5 $L.head = x$

Assuming the element inserted is not already in the table, insertion takes constant time to run.

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$)

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$)

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$)

LIST-PREPEND(L, x)

1 $x.next = L.head$

2 $x.prev = \text{NIL}$

3 **if** $L.head \neq \text{NIL}$

4 $L.head.prev = x$

5 $L.head = x$

Assuming the element inserted is not already in the table, insertion takes constant time to run.

To guarantee this, we first run a search.

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

Worst case running time is:

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$) → Proportional to list length.

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$) → Proportional to list length.

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$) → Constant.

Hashing with chaining

- When collisions are resolved with chaining, the dictionary operations must call on the linked list operations.

Worst case running time is:

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND($T[h(x.key)], x$) → Proportional to list length.

CHAINED-HASH-SEARCH(T, k)

1 **return** LIST-SEARCH($T[h(k)], k$) → Proportional to list length.

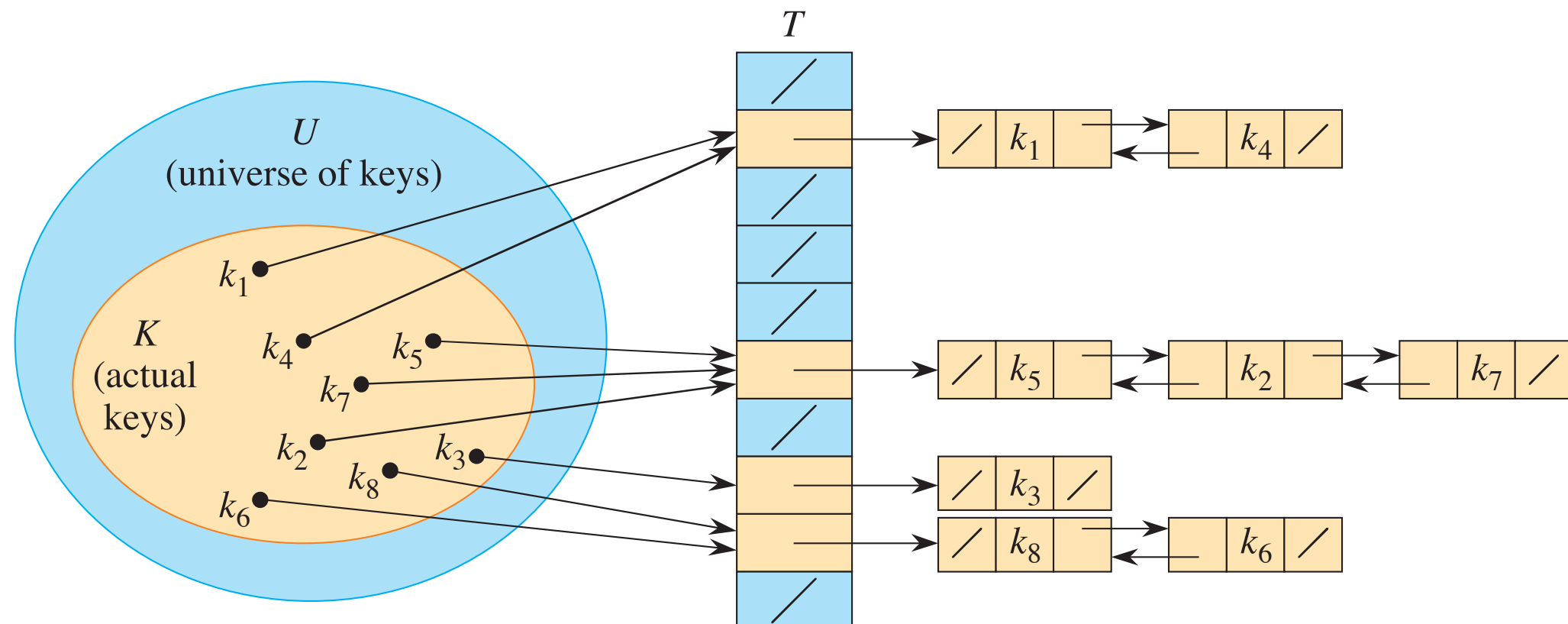
CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE($T[h(x.key)], x$) → Constant.

- To maximize the efficiency of hashing with chaining, it is best to evenly distribute collisions in the hashing table.

Exercise

- Workout what would happen if given a hashing table with 9 positions, division hashing function, and collision resolved by chaining, when inserting the keys 5, 28, 19, 15, 20, 33, 12, 17, 10.

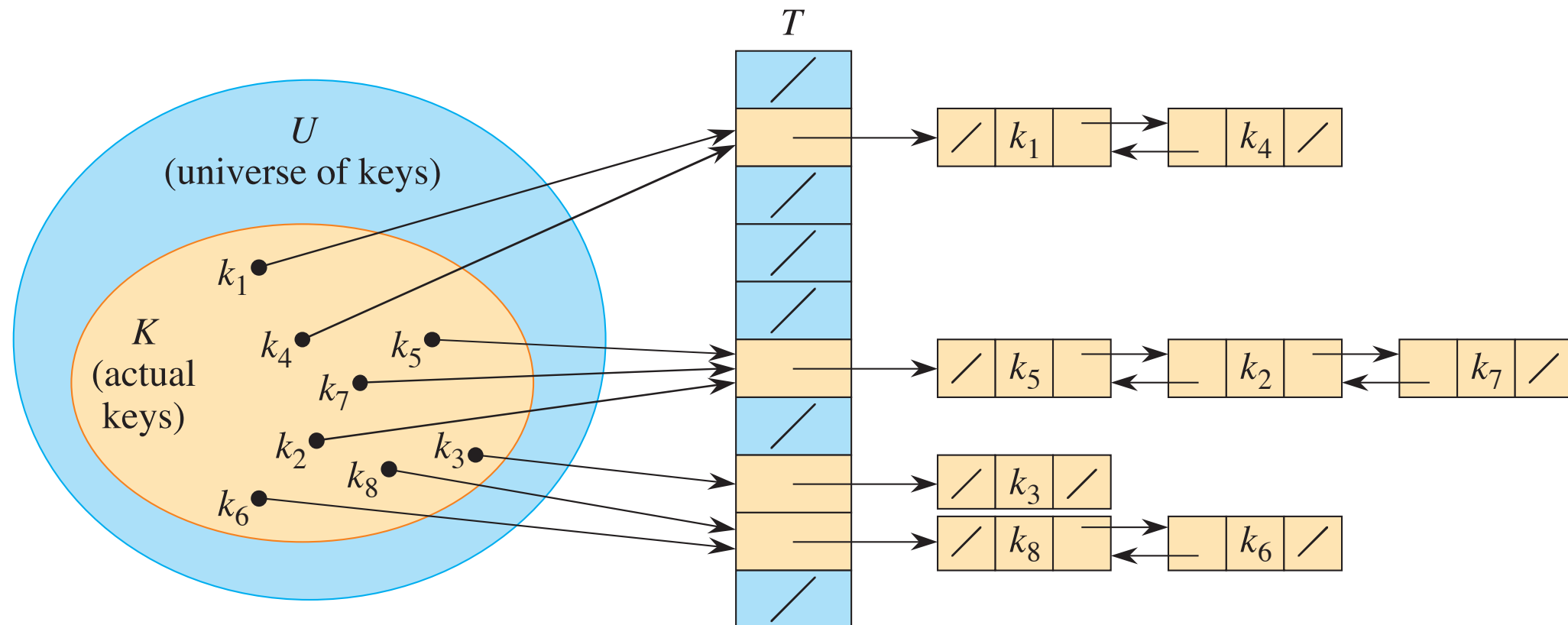


Exercise

- Workout what would happen if given a hashing table with 9 positions, division hashing function, and collision resolved by chaining, when inserting the keys 5, 28, 19, 15, 20, 33, 12, 17, 10.

The **division method** for m positions.

$$h(k) = k \bmod m$$



Exercise

- Workout what would happen if given a hashing table with 9 positions, division hashing function, and collision resolved by chaining, when inserting the keys 5, 28, 19, 15, 20, 33,12,17,10.

The **division method** for m positions.

$$h(k) = k \bmod m$$

Homework (due with Assignment 1)

- You need to store a set of n keys in a hash table of size m .

Show that if the keys are drawn from a universe U with

$$|U| > (n - 1)m,$$

then U has a subset of size n consisting of keys that all hash to the same slot. So that the worst-case searching time for hashing with chaining is $\Theta(n)$.